

# StealthGuard

## *Intrusion Detection System with File Integrity Monitoring*

Complete Guide — Features, Commands, Explanations & Viva Prep  
IIUC CSE Department | OS Lab Project | 6th Semester | Spring 2026

|                |                                                                                    |
|----------------|------------------------------------------------------------------------------------|
| Team Members   | Tahasina Tasnim Afra (C233456), Nusrat Jahan (C233464), Asliraf Samaylan (C233446) |
| Supervisor     | Ms. Sauda Adiv Hanum — Lecturer, CSE, IIUC                                         |
| Course         | Operating Systems Lab   Section 6BF                                                |
| Files          | stealthguard.sh (Shell) + verify.py (Python 3)                                     |
| Total Features | 12 Features covering Screenshot, Hashing, Email Alerts, Folder Watch               |

This document covers everything in one place — what every feature does in simple language, the exact commands to run for each feature, key word explanations, and viva answers. Share with your group before the evaluation.

## PART 1 — What is StealthGuard?

### In One Line

StealthGuard is a silent security camera for your computer. It watches the screen, records what programs are running, detects any changes, and emails you immediately if anything suspicious happens — all automatically without anyone knowing.

### Two Files — Two Layers

| File            | Language     | Role                                                                                            |
|-----------------|--------------|-------------------------------------------------------------------------------------------------|
| stealthguard.sh | Bash / Shell | Main controller — takes screenshot, saves processes, calls Python, sends emails, watches folder |
| verify.py       | Python 3     | Verifier — checks file size, calculates SHA-256 hash, compares hashes, writes audit log         |

### The Pipeline

Every time StealthGuard runs, it follows this exact order:

| Step            | What Happens                                                | Who    |
|-----------------|-------------------------------------------------------------|--------|
| 1. Trigger      | Manual (--capture) or Auto (every 10 seconds)               | Shell  |
| 2. Check Vault  | Creates /SecurityVault if missing                           | Shell  |
| 3. Screenshot   | scrot takes full desktop screenshot with timestamp filename | Shell  |
| 4. Process Snap | ps aux saves top 10 processes by CPU                        | Shell  |
| 5. Verify       | Python checks file exists and size is not 0 bytes           | Python |
| 6. Hash Check   | SHA-256 calculated and compared with previous               | Python |
| 7. Log          | Everything written to audit log permanently                 | Python |
| 8. Email        | Alert email sent if hash changed or file modified           | Shell  |

## PART 2 — Setup Commands (Run Once)

### Go to project folder

```
cd ~/stealthguard
```

*Meaning: Navigate into your project directory*

### Check files exist

```
ls
```

*Meaning: Should show stealthguard.sh and verify.py*

### Give execute permission

```
chmod +x stealthguard.sh
```

*Meaning: chmod = change mode. +x = add execute permission. Without this shell wont run*

### Create SecurityVault

```
sudo mkdir /SecurityVault  
sudo chmod 777 /SecurityVault
```

*Meaning: mkdir = make directory. 777 = full permission for everyone*

### Install all tools

```
sudo apt update  
sudo apt install scrot mutt zip inotify-tools -y
```

*Meaning: scrot=screenshot, mutt=email, zip=compress, inotify=folder watch, -y=yes to all*

### Check Gmail config

```
cat ~/.muttrc
```

*Meaning: Should show your email and app password*

### Clear old files

```
rm -rf /SecurityVault/*
```

*Meaning: rm=remove. -rf=force delete all. \*=everything inside*

### Confirm vault empty

```
ls /SecurityVault/
```

*Meaning: Should show nothing — clean start*

## PART 3 — All Features: Explanation + Commands

### Feature 1 Automated Screenshot Capture

[Shell]

#### Simple Meaning:

Takes a photo of whatever is on the computer screen and saves it as a PNG image file inside /SecurityVault/ automatically.

#### Real World:

Like a CCTV camera taking a picture every few seconds silently without anyone knowing.

#### Key Words:

**scrot** — The tool that takes the screenshot. Like a camera app but for the whole screen

**\$FILENAME** — The full path where the screenshot gets saved

**\$?** — Exit code of last command. 0 = success, anything else = failure

**return 1** — Exit the function with an error code

#### Code (stealthguard.sh):

```
scrot "$FILENAME"

if [ $? -eq 0 ]; then
echo "[OK] Screenshot saved"
else
echo "[ERROR] Screenshot failed"
return 1
fi
```

#### Commands to show ma'am:

```
./stealthguard.sh --capture
ls /SecurityVault/
```

### Feature 2 Timestamp-Based File Naming

[Shell]

#### Simple Meaning:

Every file gets a unique name based on the exact date and time it was captured. Example: capture\_2026-06-24\_00-17-28.png. This prevents any two files from overwriting each other.

#### Real World:

Like how WhatsApp names photos IMG-20260624-001.jpg automatically so nothing is lost.

#### Key Words:

**date** — Linux command that gives current date and time

**TIMESTAMP** — Variable storing the formatted date and time

**\$()** — Run a command and store its result in a variable

**+"%Y-%m-%d\_%H-%M-%S"** — Format: Year-Month-Day\_Hour-Minute-Second

#### Code (stealthguard.sh):

```
TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S")
FILENAME="$VAULT/capture_$TIMESTAMP.png"
PROC_FILE="$VAULT/processes_$TIMESTAMP.txt"
```

#### Commands to show ma'am:

```
ls /SecurityVault/
```

### Feature 3      Process Snapshot Monitoring

[Shell]

#### Simple Meaning:

Every time a screenshot is taken, the system also saves a list of the top 10 programs running on the computer at that exact moment, sorted by CPU usage.

#### Real World:

Like checking Task Manager on Windows — but it saves the list automatically as evidence with every single capture. If a hacker was running something, it shows up here.

#### Key Words:

- ps aux** — List all running programs/processes on the system
- sort=-%cpu** — Sort by highest CPU usage first
- head -11** — Take only first 11 lines (1 header + 10 processes)
- >>** — Append to file — adds without deleting old content

#### Code (stealthguard.sh):

```
echo "=== Top 10 Processes at $TIMESTAMP ===" > "$PROC_FILE"
ps aux --sort=-%cpu | head -11 >> "$PROC_FILE"
```

#### Commands to show ma'am:

```
cat /SecurityVault/processes_*.txt
```

**Simple Meaning:**

Before accepting any screenshot as evidence, Python checks if the file is empty (0 bytes). If the capture failed and saved nothing, it gets rejected immediately and flagged as corrupt.

**Real World:**

Like a printer checking if the ink is empty before printing. If empty, it refuses and warns you instead of giving you a blank page as fake proof.

**Key Words:**

**os.path.getsize()** — Python function that returns file size in bytes

**0 bytes** — Completely empty file — nothing was saved

**return False** — Stop the function and report failure

**log(msg)** — Write the failure message to audit log

**Code (verify.py):**

```
size = os.path.getsize(screenshot)
if size == 0:
    msg = f"FAILURE - Zero-byte file (corrupt): {filename}"
    print(f"{RED}[FAIL] {msg}{NC}")
    log(msg)
    return False
```

**Commands to show ma'am:**

```
touch /SecurityVault/fake_capture.png
python3 verify.py /SecurityVault/fake_capture.png
/SecurityVault/processes_*.txt
```

**Simple Meaning:**

Every screenshot gets converted into a unique 64-character code called a hash. Think of it like a fingerprint for the image. No two different images will ever produce the same hash.

**Real World:**

Like a fingerprint scanner. Every person has a unique fingerprint. Every screenshot has a unique hash. Even one pixel different = completely different hash.

**Key Words:**

**SHA-256** — Secure Hash Algorithm — converts any file into a fixed 64-character code

**hashlib** — Python library that does the hashing calculation

**"rb"** — Read binary mode — reads image file as raw data not text

**f.read(4096)** — Read file in chunks of 4096 bytes at a time — efficient for large files

**hexdigest()** — Returns the final hash as a readable string like 66362731d9...

#### Code (verify.py):

```
def get_sha256(filepath):  
    sha256 = hashlib.sha256()  
    with open(filepath, "rb") as f:  
        for chunk in iter(lambda: f.read(4096), b""):  
            sha256.update(chunk)  
    return sha256.hexdigest()
```

#### Commands to show me'am:

```
cat /SecurityVault/.previous_hash
```

**Simple Meaning:**

After calculating the new screenshot hash, Python compares it with the hash saved from the last capture. If different — ALERT. Even one pixel change triggers it.

**Real World:**

Like comparing two fingerprints. Match = same person = nothing changed. No match = different person = something is wrong.

**3 Outcomes:**

- FIRST RUN — No previous hash exists. Just saves current hash for next time.
- NO CHANGE — Hashes match. Screen identical. No alert.
- ALERT — Hashes different. Screen changed. Email sent automatically.

**Key Words:**

**.previous\_hash** — Hidden file storing the last hash for comparison

**None** — Python word for nothing/empty — means no previous hash exists yet

**load\_previous\_hash()** — Function that reads the saved hash from file

**save\_hash()** — Function that saves current hash for next comparison

**Code (verify.py):**

```
if previous_hash is None:
    status = "FIRST RUN - No previous hash."
elif current_hash == previous_hash:
    status = "NO CHANGE - Screen identical."
else:
    status = "ALERT - Screen change detected!"

save_hash(current_hash)
```

**Commands to show ma'am:**

```
rm -f /SecurityVault/.previous_hash
./stealthquard.sh --capture
qterminal &
./stealthquard.sh --capture
cat /SecurityVault/system_security_audit.log
```

**Simple Meaning:**

Every event — screenshot name, file size, SHA-256 hash, change status, and full process list — gets written permanently to system\_security\_audit.log with exact timestamps. This file never gets deleted or overwritten, only added to.

**Real World:**



Like a security guard's notebook. Every incident written down with time and details. Nothing ever erased. You can always go back and check exactly what happened.

**Key Words:**

**open(LOG, "a")** — Open file in append mode — adds to end, never deletes

**datetime.now()** — Get current date and time in Python

**strftime()** — Format the date as readable text

**"=" \* 50** — Print = sign 50 times — divider between log entries

**for line in processes.splitlines()** — Split process text into lines and log each one

**Code (verify.py):**

```
def log(message):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(LOG_FILE, "a") as f:
        f.write(f"[{timestamp}] {message}\n")
```

**Commands to show ma'am:**

```
cat /SecurityVault/system_security_audit.log
ls -la /SecurityVault/
```

**Simple Meaning:**

Instead of waiting for auto mode, you can type `--capture` after the command to trigger one single capture immediately on demand.

**Real World:**

Like a security guard who does automatic rounds every hour but can also do an immediate check anytime someone calls.

**Key Words:**

**\$1** — First argument you type after the command name

**[ "\$1" == "--capture" ]** — Check if what you typed equals `--capture`

**do\_capture** — Call the capture function

**Code (stealthguard.sh):**

```
if [ "$1" == "--capture" ]; then
echo -e "${YELLOW}[MODE] Manual capture.${NC}"
do_capture
fi
```

**Commands to show ma'am:**

```
./stealthguard.sh --capture
```

**Simple Meaning:**

Running the script with no extra arguments starts auto mode. It captures every 10 seconds forever on its own until you press `Ctrl+C`.

**Real World:**

Like a CCTV camera that records 24/7 automatically without anyone pressing record.

**Key Words:**

**while true** — Loop that runs forever — no stopping condition

**do / done** — Start and end of the loop body

**sleep \$INTERVAL** — Pause for 10 seconds before next capture

**Ctrl+C** — Sends SIGINT signal to stop the program

**Code (stealthguard.sh):**

```
while true; do
do_capture
echo "[INFO] Waiting ${INTERVAL} seconds..."
sleep $INTERVAL
done
```

**Commands to show ma'am:**

```
./stealthguard.sh  
# Let it run 2-3 cycles then press Ctrl+C
```

**Simple Meaning:**

Compresses all screenshots, process files, and the audit log into one ZIP file and emails it to the configured email address as an attachment.

**Real World:**

Like a security company emailing the entire week's CCTV footage to the building owner as one compressed file.

**Key Words:**

**zip -j** — Create a zip file. -j = dont include folder paths, just the files

**mutt** — Command line email tool — like Gmail but in the terminal

**-a "\$ZIPFILE"** — Attach the zip file to the email

**/tmp/** — Temporary folder in Linux for files not needed permanently

**Code (stealthguard.sh):**

```
ZIPFILE="/tmp/stealthguard_$(date +"%Y-%m-%d_%H-%M-%S").zip"

zip -j "$ZIPFILE" "$VAULT"/capture_*.png \
"$VAULT"/processes_*.txt \
"$VAULT"/system_security_audit.log

echo "Report..." | mutt -s "StealthGuard Report" \
-a "$ZIPFILE" -- "$TO_EMAIL"
```

**Commands to show ma'am:**

```
./stealthguard.sh --sendemail
ls /tmp/stealthguard_*.zip
```

**Simple Meaning:**

When Python detects the hash changed (screen changed), the shell automatically catches that ALERT in the output and sends an email immediately — no extra input needed.

**Real World:**

Like a burglar alarm that not only rings but also calls the owner's phone automatically.

**Key Words:**

**PYTHON\_OUTPUT=\$()** — Run Python and store everything it printed as a variable

**grep -q "ALERT"** — Search for the word ALERT silently. -q = quiet no output

**awk '{print \$2}'** — Print the second word of a line — extracts the hash value

**send\_alert\_email()** — Function that sends the alert email using mutt

**Code (stealthguard.sh):**

```
PYTHON_OUTPUT=$(python3 "$VERIFY_SCRIPT" "$FILENAME" "$PROC_FILE")
echo "$PYTHON_OUTPUT"

if echo "$PYTHON_OUTPUT" | grep -q "ALERT"; then
HASH=$(echo "$PYTHON_OUTPUT" | grep "SHA-256" | awk '{print $2}')
send_alert_email "$FILENAME" "$HASH" "$TIMESTAMP"
fi
```

Commands to show me'am:

```
./stealthguard.sh --capture
qterminal &
./stealthguard.sh --capture
# Check phone for alert email
```

**Simple Meaning:**

Watches the SecurityVault folder 24/7 in real time. The moment anyone touches, changes, or deletes any file inside it — the system detects it instantly and sends an email alert. No waiting for next capture cycle.

**Real World:**

Like a pressure sensor under a rug. The moment someone steps on it, the alarm goes off instantly — not after 10 seconds, right away.

**Key Words:**

**inotifywait** — Linux tool that watches a folder for changes in real time

**-m** — Monitor mode — keeps watching forever, never stops

**-e modify,create,delete** — Which events to watch for

**--format '%e %f'** — Print event type and filename when something happens

**while read -r event filename** — Read each detected event one by one

**continue** — Skip this loop cycle — used to ignore the log file itself

**local** — Variable only exists inside that function

**Code (stealthguard.sh):**

```
inotifywait -m "$VAULT" -e modify,create,delete \
--format '%e %f' |
while read -r event filename; do
if [ "$filename" == "system_security_audit.log" ]; then
continue
fi
echo "[ALERT] $event on: $filename"
send_modification_email "$filename" "$event"
done
```

**Commands to show ma'am:**

```
# Terminal 1
./stealthguard.sh --watchfolder
# Terminal 2
echo "tampered" >> /SecurityVault/capture_*.png
# Check phone for modification alert email
```

## PART 4 — Full Command List (All Features in Order)

### SETUP

```
cd ~/stealthguard
ls
chmod +x stealthguard.sh
sudo mkdir /SecurityVault
sudo chmod 777 /SecurityVault
sudo apt install scrot mutt zip inotify-tools -y
cat ~/.muttrc
rm -rf /SecurityVault/*
```

### FEATURE 1 + 2 + 3 — Capture, Timestamp, Processes

```
./stealthguard.sh --capture
ls /SecurityVault/
cat /SecurityVault/processes_*.txt
```

### FEATURE 4 — Zero Byte Detection

```
touch /SecurityVault/fake_capture.png
python3 verify.py /SecurityVault/fake_capture.png
/SecurityVault/processes_*.txt
```

### FEATURE 5 + 6 — SHA-256 Hash + Change Detection

```
rm -f /SecurityVault/.previous_hash
./stealthguard.sh --capture
qterminal &
./stealthguard.sh --capture
cat /SecurityVault/.previous_hash
```

### FEATURE 7 — Audit Log

```
cat /SecurityVault/system_security_audit.log
ls -la /SecurityVault/
```

### FEATURE 8 — Manual Trigger

```
./stealthguard.sh --capture
```

### FEATURE 9 — Auto Loop Mode

```
./stealthguard.sh
# Wait 2-3 cycles then Ctrl+C
```

## FEATURE 10 — Zip and Email

```
./stealthguard.sh --sendemail  
ls /tmp/stealthguard_*.zip
```

## FEATURE 11 — Alert Email on Hash Change

```
./stealthguard.sh --capture  
qterminal &  
./stealthguard.sh --capture  
# Check phone for alert email
```

## FEATURE 12 — Folder Watch + Modification Email

```
# Open Terminal 1  
./stealthguard.sh --watchfolder  
# Open Terminal 2  
echo "tampered" >> /SecurityVault/capture_*.png  
# Check phone for modification email
```



## PART 5 — Viva Questions & Simple Answers

| <b>What is StealthGuard?</b>                     | A silent security monitoring system for Kali Linux using Shell scripting and Python       |
|--------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>What does scrot do?</b>                       | Takes a screenshot of the full desktop and saves it as a PNG file                         |
| <b>What is SHA-256?</b>                          | A mathematical formula that converts any file into a unique 64-character fingerprint code |
| <b>What is .previous_hash?</b>                   | A hidden file that stores the last SHA-256 hash for comparison in the next capture cycle  |
| <b>What is append mode?</b>                      | File writing mode that adds to the end without deleting old data — audit log uses this    |
| <b>What does ps aux do?</b>                      | Lists all running processes on the system with CPU usage, memory, and name                |
| <b>What is inotifywait?</b>                      | A Linux tool that watches a folder in real time and triggers when any file changes        |
| <b>What is mutt?</b>                             | A command line email tool used to send emails including attachments from the terminal     |
| <b>What does \$1 mean?</b>                       | The first argument typed after the command. --capture in ./stealthguard.sh --capture      |
| <b>What does \$? mean?</b>                       | Exit code of last command. 0 = success, 1 = failure                                       |
| <b>Why two files?</b>                            | Shell is best for OS-level tasks like screenshots. Python is best for hashing and logging |
| <b>What is zero-byte check?</b>                  | Checks if the captured file is empty before accepting it as valid evidence                |
| <b>What is while true?</b>                       | An infinite loop that runs forever until stopped with Ctrl+C                              |
| <b>What is sys.argv?</b>                         | A list in Python of arguments passed from the shell script                                |
| <b>What is grep?</b>                             | A search command. grep -q ALERT searches for the word ALERT silently in output            |
| <b>What is the audit log?</b>                    | A permanent file that records every event with timestamp, hash, size and processes        |
| <b>Is --capture the only way?</b>                | No. Auto mode runs every 10 seconds automatically. --capture is just for demo             |
| <b>What is inter-process communication here?</b> | Shell passes screenshot path to Python using arguments. Python reads via sys.argv         |
| <b>Why sort processes by CPU?</b>                | Malicious programs usually use high CPU. Sorting puts suspicious ones at the top          |
| <b>What is /SecurityVault?</b>                   | The main storage folder where all captures, logs, and process files are saved             |

## PART 6 — Quick Cheatsheet

### All Commands at a Glance

| Command                                                   | What It Does                       |
|-----------------------------------------------------------|------------------------------------|
| <code>./stealthguard.sh --capture</code>                  | Manual single capture              |
| <code>./stealthguard.sh</code>                            | Auto mode every 10 seconds         |
| <code>./stealthguard.sh --sendemail</code>                | Zip all captures and email         |
| <code>./stealthguard.sh --watchfolder</code>              | Watch folder for modifications     |
| <code>./stealthguard.sh --help</code>                     | Show all available commands        |
| <code>cat /SecurityVault/system_security_audit.log</code> | View full audit log                |
| <code>cat /SecurityVault/processes_*.txt</code>           | View process snapshots             |
| <code>cat /SecurityVault/previous_hash</code>             | View stored SHA-256 hash           |
| <code>ls -la /SecurityVault/</code>                       | List all files including hidden    |
| <code>rm -rf /SecurityVault/*</code>                      | Clear all captures for fresh start |

### OS Concepts Covered

| OS Concept                  | Where in Project                                                      |
|-----------------------------|-----------------------------------------------------------------------|
| Process Management          | <code>ps aux</code> monitors all running processes                    |
| File System                 | Creating, reading, writing files in <code>/SecurityVault</code>       |
| Shell Scripting             | Entire <code>stealthguard.sh</code> — functions, loops, conditions    |
| Inter-Process Communication | Shell calls Python with arguments, Python reads <code>sys.argv</code> |
| Cryptographic Hashing       | SHA-256 in <code>verify.py</code> using <code>hashlib</code> library  |
| File Integrity Monitoring   | Zero-byte check and hash comparison every cycle                       |
| Persistent Logging          | Append mode log file that never overwrites                            |
| Signal Handling             | <code>Ctrl+C</code> stops the auto loop (SIGINT signal)               |
| Real-Time Event Monitoring  | <code>inotifywait</code> watches folder for live changes              |
| Email Automation            | <code>mutt</code> sends alerts automatically from terminal            |